

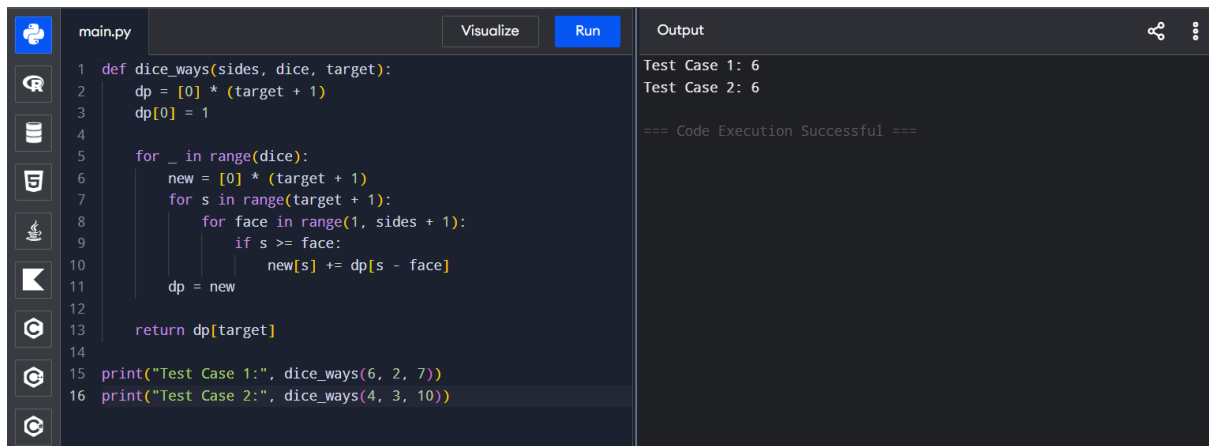
CHAPTER – 4

Name: AKSHAYA A

Regno: 192421222

Course: CSA0603– DAA

EXPERIMENT – 1

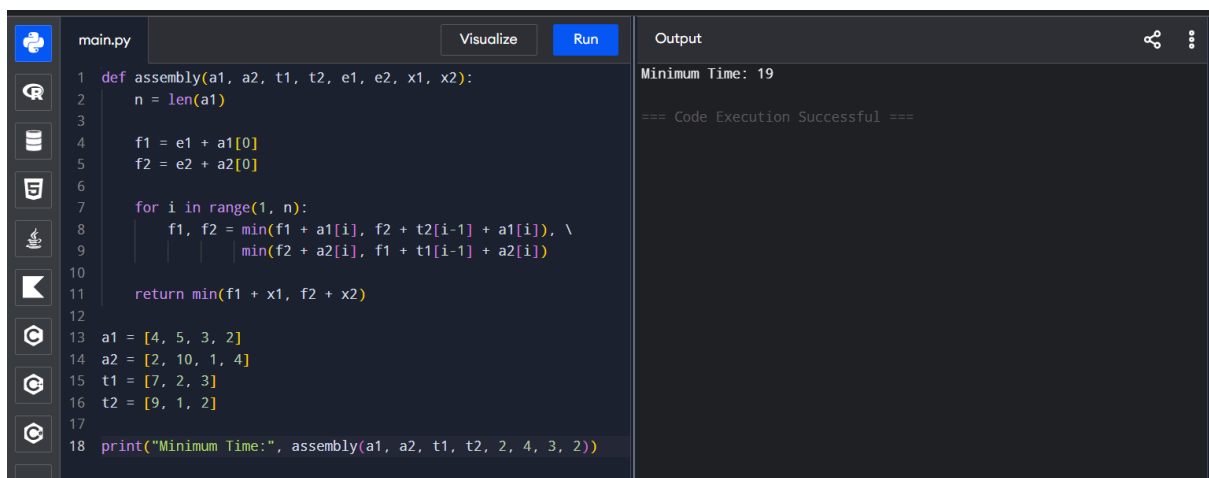


```
main.py Visualize Run Output
1 def dice_ways(sides, dice, target):
2     dp = [0] * (target + 1)
3     dp[0] = 1
4
5     for _ in range(dice):
6         new = [0] * (target + 1)
7         for s in range(target + 1):
8             for face in range(1, sides + 1):
9                 if s >= face:
10                    new[s] += dp[s - face]
11         dp = new
12
13     return dp[target]
14
15 print("Test Case 1:", dice_ways(6, 2, 7))
16 print("Test Case 2:", dice_ways(4, 3, 10))

Test Case 1: 6
Test Case 2: 6

=== Code Execution Successful ===
```

EXPERIMENT – 2



```
main.py Visualize Run Output
1 def assembly(a1, a2, t1, t2, e1, e2, x1, x2):
2     n = len(a1)
3
4     f1 = e1 + a1[0]
5     f2 = e2 + a2[0]
6
7     for i in range(1, n):
8         f1, f2 = min(f1 + a1[i], f2 + t2[i-1] + a1[i]), \
9                 min(f2 + a2[i], f1 + t1[i-1] + a2[i])
10
11     return min(f1 + x1, f2 + x2)
12
13 a1 = [4, 5, 3, 2]
14 a2 = [2, 10, 1, 4]
15 t1 = [7, 2, 3]
16 t2 = [9, 1, 2]
17
18 print("Minimum Time:", assembly(a1, a2, t1, t2, 2, 4, 3, 2))

Minimum Time: 19

=== Code Execution Successful ===
```

EXPERIMENT – 3

main.py

Visualize

Run

1

def assembly(lines, transfer):

2

n = len(lines[0])

3

dp = lines[0][:]

4

for i in range(1, n):

5

new = []

6

for j in range(3):

7

best = min(dp[k] + transfer[k][j] for k in range

8

(3))

9

new.append(best + lines[j][i])

10

dp = new

11

return min(dp)

12

lines = [

13

[5, 9, 3],

14

[6, 8, 4],

15

[7, 6, 5]

16

]

17

transfer = [

18

[0, 2, 3],

19

[2, 0, 4],

20

[3, 4, 0]

21

]

22

print("Minimum Production Time:", assembly(lines, transfer))

Output

Minimum Production Time: 14

=== Code Execution Successful ===

EXPERIMENT – 4

main.py

Visualize

Run

1

from itertools import permutations

2

3

def tsp(a):

4

best = float('inf')

5

for p in permutations(range(1, len(a))):

6

path = (0,) + p + (0,)

7

cost = sum(a[path[i]][path[i+1]] for i in range(len

8

(path)-1))

9

best = min(best, cost)

10

return best

11

tests = [

12

[[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]],

13

[[0,10,10,10],[10,0,10,10],[10,10,0,10],[10,10,10,0]],

14

[[0,1,2,3],[1,0,4,5],[2,4,0,6],[3,5,6,0]]

15

]

16

17

for a in tests:

18

print("Minimum Path Distance:", tsp(a))

Output

Minimum Path Distance: 80

Minimum Path Distance: 40

Minimum Path Distance: 14

=== Code Execution Successful ===

EXPERIMENT – 5

main.py

Visualize

Run

1

from itertools import permutations

2

3

d = {

4

'A':{'B':10,'C':15,'D':20,'E':25},

5

'B':{'A':10,'C':35,'D':25,'E':30},

6

'C':{'A':15,'B':35,'D':30,'E':20},

7

'D':{'A':20,'B':25,'C':30,'E':15},

8

'E':{'A':25,'B':30,'C':20,'D':15}

9

}

10

best = float('inf')

11

route = None

12

13

for p in permutations('BCDE'):

14

r = 'A' + ''.join(p) + 'A'

15

cost = sum(d[r[i]][r[i+1]] for i in range(5))

16

if cost < best:

17

best, route = cost, r

18

19

print("Shortest Route:", " -> ".join(route))

20

print("Total Distance:", best)

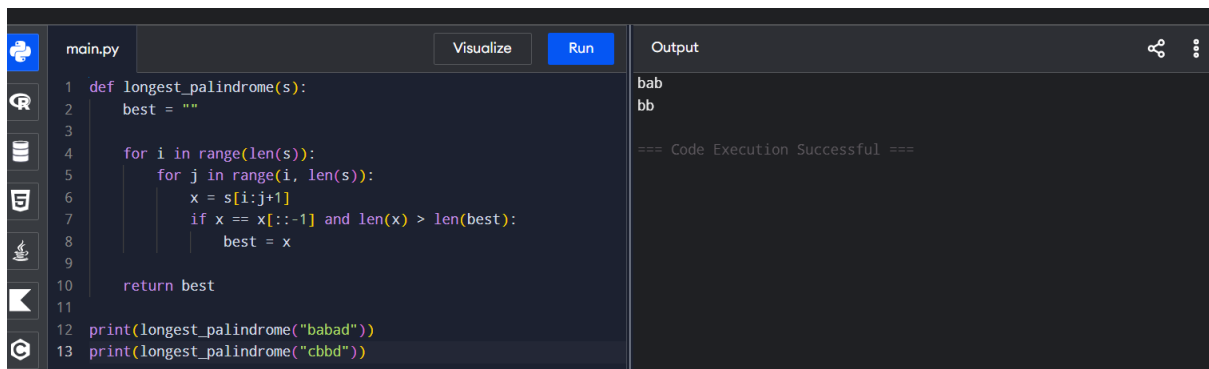
Output

Shortest Route: A -> B -> D -> E -> C -> A

Total Distance: 85

=== Code Execution Successful ===

EXPERIMENT – 6



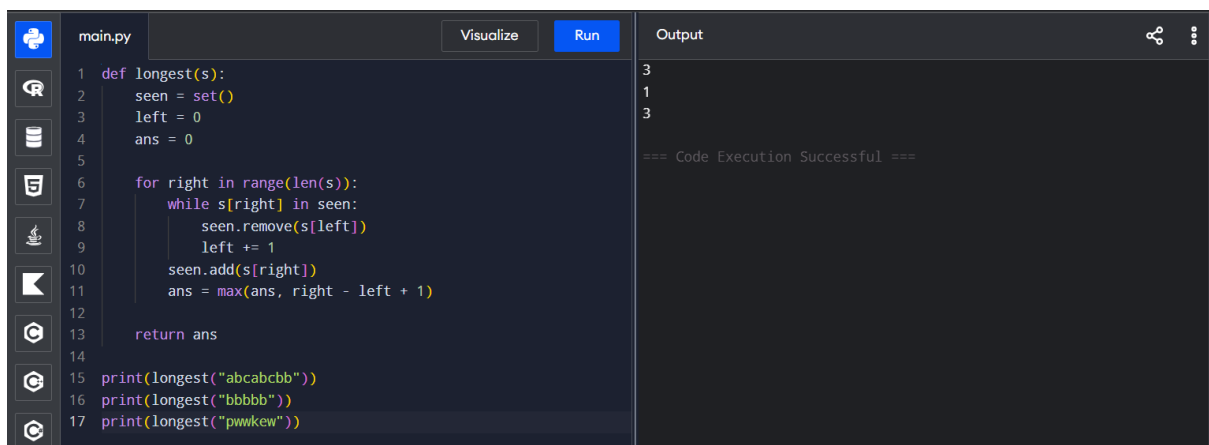
The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'longest_palindrome(s)' that finds the longest palindrome in a string 's' by checking all possible substrings. It prints the results for 'babad' and 'cbbd'. The output window shows 'bab' and 'bb' as the longest palindromes, followed by a success message.

```
1 def longest_palindrome(s):
2     best = ""
3
4     for i in range(len(s)):
5         for j in range(i, len(s)):
6             x = s[i:j+1]
7             if x == x[::-1] and len(x) > len(best):
8                 best = x
9
10    return best
11
12 print(longest_palindrome("babad"))
13 print(longest_palindrome("cbbd"))
```

Output:

```
bab
bb
=== Code Execution Successful ===
```

EXPERIMENT – 7



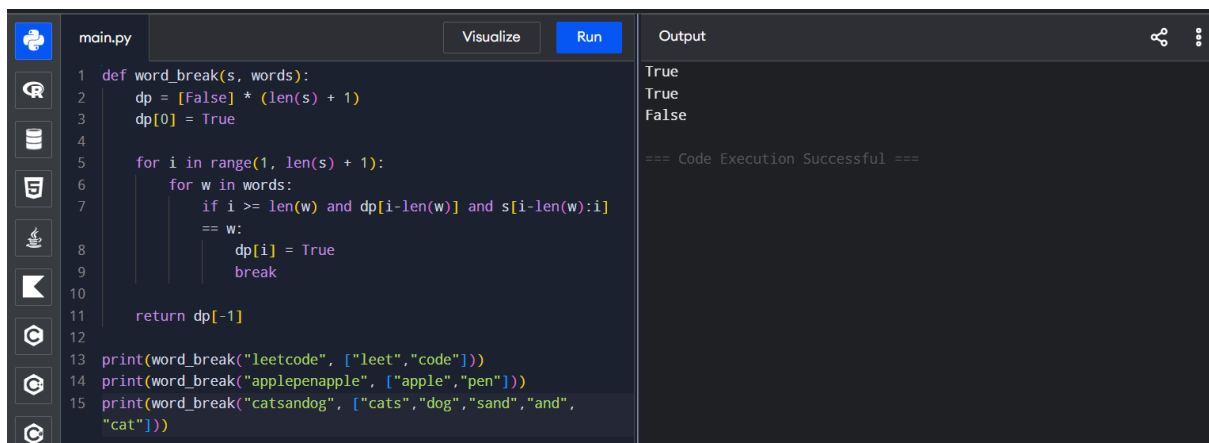
The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'longest(s)' that finds the longest palindromic substring using a set to track seen substrings. It prints the results for 'abcabcbb', 'bbbbbb', and 'pwwkew'. The output window shows the lengths of the longest palindromic substrings: 3, 1, and 3, followed by a success message.

```
1 def longest(s):
2     seen = set()
3     left = 0
4     ans = 0
5
6     for right in range(len(s)):
7         while s[right] in seen:
8             seen.remove(s[left])
9             left += 1
10        seen.add(s[right])
11        ans = max(ans, right - left + 1)
12
13    return ans
14
15 print(longest("abcabcbb"))
16 print(longest("bbbbbb"))
17 print(longest("pwwkew"))
```

Output:

```
3
1
3
=== Code Execution Successful ===
```

EXPERIMENT – 8



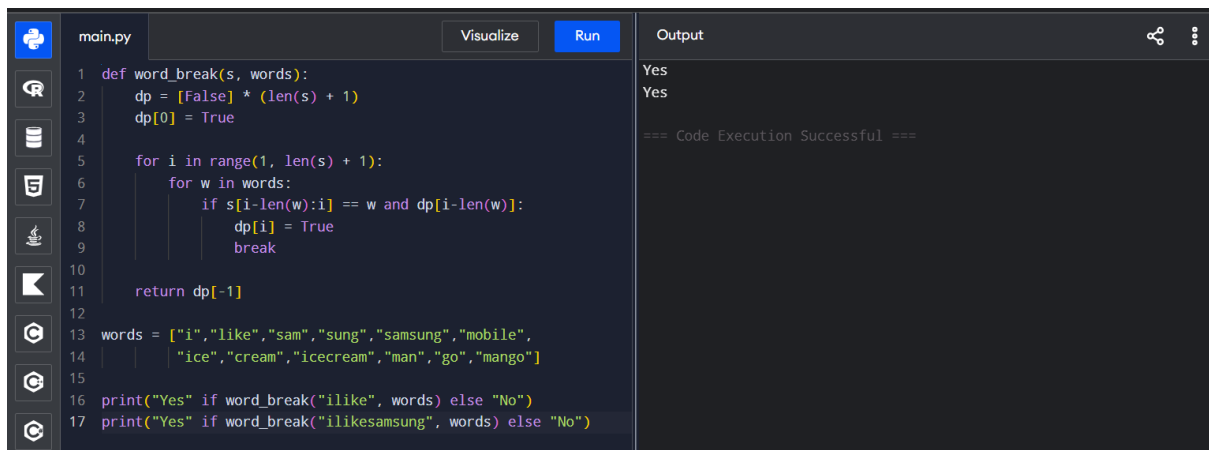
The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'word_break(s, words)' that checks if a string 's' can be broken down into words from a list 'words' using dynamic programming. It prints the results for 'leetcode', 'applepenapple', and 'catsandog'. The output window shows 'True', 'True', and 'False' as the results, followed by a success message.

```
1 def word_break(s, words):
2     dp = [False] * (len(s) + 1)
3     dp[0] = True
4
5     for i in range(1, len(s) + 1):
6         for w in words:
7             if i >= len(w) and dp[i-len(w)] and s[i-len(w):i] == w:
8                 dp[i] = True
9                 break
10
11    return dp[-1]
12
13 print(word_break("leetcode", ["leet", "code"]))
14 print(word_break("applepenapple", ["apple", "pen"]))
15 print(word_break("catsandog", ["cats", "dog", "sand", "and", "cat"]))
```

Output:

```
True
True
False
=== Code Execution Successful ===
```

EXPERIMENT – 9

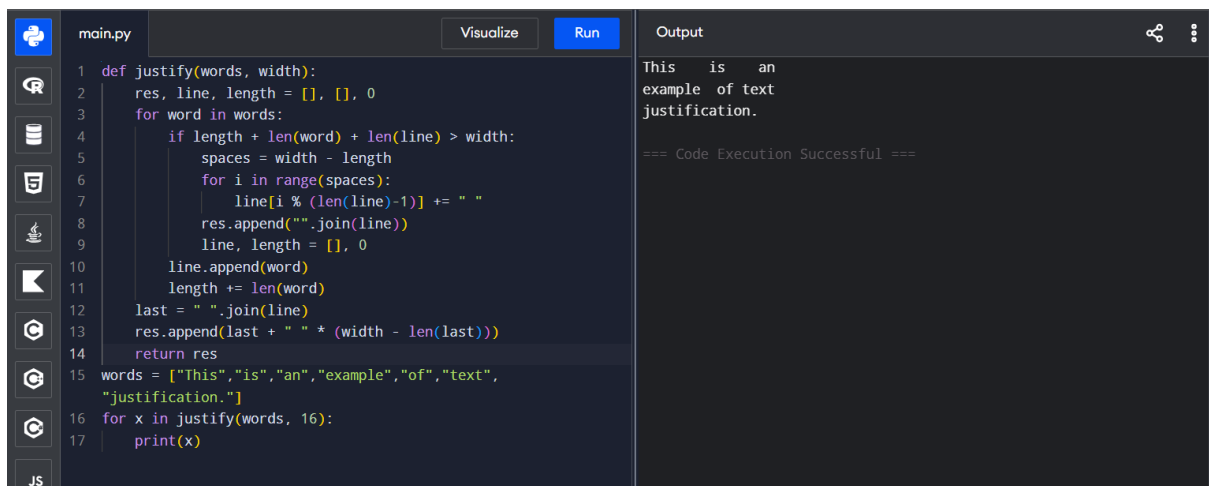


The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code defines a function 'word_break(s, words)' that uses a dynamic programming approach to determine if a string 's' can be segmented into words from the list 'words'. The function returns a boolean value. The code then tests the function with 'ilike' and 'ilikesamsung'.

```
1 def word_break(s, words):
2     dp = [False] * (len(s) + 1)
3     dp[0] = True
4
5     for i in range(1, len(s) + 1):
6         for w in words:
7             if s[i-len(w):i] == w and dp[i-len(w)]:
8                 dp[i] = True
9                 break
10
11     return dp[-1]
12
13 words = ["i", "like", "sam", "sung", "samsung", "mobile",
14         "ice", "cream", "icecream", "man", "go", "mango"]
15
16 print("Yes" if word_break("ilike", words) else "No")
17 print("Yes" if word_break("ilikesamsung", words) else "No")
```

The output panel shows the results of the execution: 'Yes' for 'ilike' and 'Yes' for 'ilikesamsung', followed by '=== Code Execution Successful ==='.

EXPERIMENT – 10

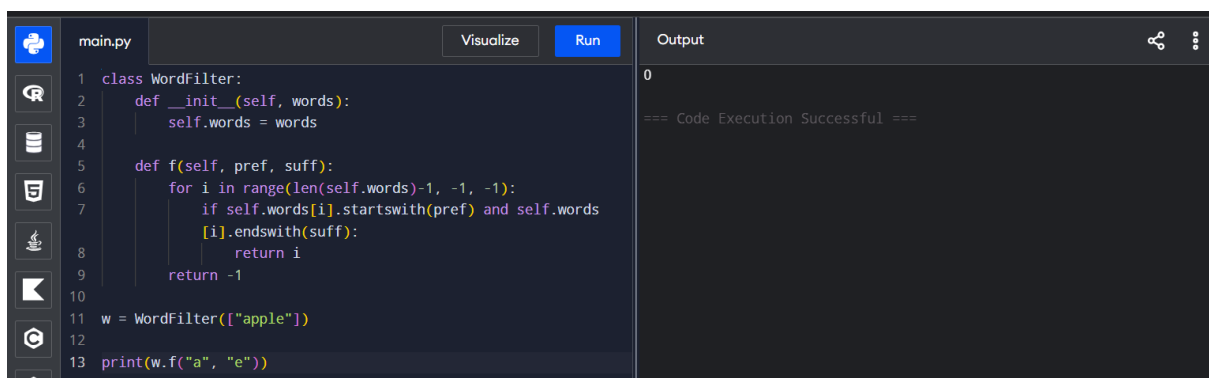


The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code defines a function 'justify(words, width)' that implements the Knuth algorithm for text justification. It takes a list of words and a target width, and returns a list of justified lines. The code then tests the function with a list of words and a width of 16.

```
1 def justify(words, width):
2     res, line, length = [], [], 0
3     for word in words:
4         if length + len(word) + len(line) > width:
5             spaces = width - length
6             for i in range(spaces):
7                 line[i % (len(line)-1)] += " "
8             res.append("".join(line))
9             line, length = [], 0
10        line.append(word)
11        length += len(word)
12    last = " ".join(line)
13    res.append(last + " " * (width - len(last)))
14    return res
15 words = ["This", "is", "an", "example", "of", "text",
16         "justification."]
17 for x in justify(words, 16):
18     print(x)
```

The output panel shows the justified text: 'This is an example of text justification.', followed by '=== Code Execution Successful ==='.

EXPERIMENT – 11



The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code defines a class 'WordFilter' that takes a list of words and provides a method 'f(pref, suff)' to find words that start with 'pref' and end with 'suff'. The code then creates an instance of 'WordFilter' with the word 'apple' and tests the 'f' method with prefixes 'a' and 'e'.

```
1 class WordFilter:
2     def __init__(self, words):
3         self.words = words
4
5     def f(self, pref, suff):
6         for i in range(len(self.words)-1, -1, -1):
7             if self.words[i].startswith(pref) and self.words[i].endswith(suff):
8                 return i
9         return -1
10
11 w = WordFilter(["apple"])
12
13 print(w.f("a", "e"))
```

The output panel shows the result of the execution: '0', followed by '=== Code Execution Successful ==='.

EXPERIMENT – 12

main.py

Visualize

Run

1

INF = 999

2

d = [

3

[0,1,5,INF,INF,INF],

4

[1,0,2,1,INF,INF],

5

[5,2,0,INF,3,INF],

6

[INF,1,INF,0,1,6],

7

[INF,INF,3,1,0,2],

8

[INF,INF,INF,6,2,0]

9

]

10

print("Before:")

11

for r in d:

12

print(r)

13

for k in range(6):

14

for i in range(6):

15

for j in range(6):

16

d[i][j] = min(d[i][j], d[i][k] + d[k][j])

17

print("\nAfter:")

18

for r in d:

19

print(r)

Output

Before:

[0, 3, 8, -4]

[999, 0, 4, 1]

[2, 999, 0, 999]

[999, 6, -5, 0]

After:

[-7, -4, -9, -11]

[-2, 0, -4, -6]

[-5, -2, -7, -9]

[-10, -7, -12, -14]

City 1 to City 3 = -9

=== Code Execution Successful ===

EXPERIMENT – 13

main.py

Visualize

Run

1

INF = 999

2

d = [

3

[0,1,5,INF,INF,INF],

4

[1,0,2,1,INF,INF],

5

[5,2,0,INF,3,INF],

6

[INF,1,INF,0,1,6],

7

[INF,INF,3,1,0,2],

8

[INF,INF,INF,6,2,0]

9

]

10

def floyd(d):

11

for k in range(6):

12

for i in range(6):

13

for j in range(6):

14

d[i][j] = min(d[i][j], d[i][k] + d[k][j])

15

return d

16

print("Before failure:", floyd([r[:] for r in d])[0][5])

17

d[1][3] = d[3][1] = INF

18

print("After failure:", floyd(d)[0][5])

Output

Before failure: 5

After failure: 8

=== Code Execution Successful ===

EXPERIMENT – 14

main.py

Visualize

Run

1

INF = 999

2

def floyd(d):

3

n = len(d)

4

for k in range(n):

5

for i in range(n):

6

for j in range(n):

7

d[i][j] = min(d[i][j], d[i][k] + d[k][j])

8

return d

9

d1 = [

10

[0, INF, 3, INF],

11

[2, 0, INF, INF],

12

[INF, 7, 0, 1],

13

[6, INF, INF, 0]

14

]

15

floyd(d1)

16

print("C to A =", d1[2][0])

17

d2 = [

18

[0, 4, INF, 5, INF],

19

[INF, 0, 1, INF, 6],

20

[2, INF, 0, 3, INF],

21

[INF, INF, 1, 0, 2],

22

[1, INF, INF, 4, 0]

23

]

Output

C to A = 7

E to C = 5

=== Code Execution Successful ===

EXPERIMENT – 15

```
main.py Visualize Run
```

```
1 k=['A','B','C','D']; f=[1,2,4,3]; n=4
2 c=[[0]*n for _ in k]; r=[[0]*n for _ in k]
3
4 for i in range(n): c[i][i]=f[i]; r[i][i]=i
5
6 for l in range(2,n+1):
7     for i in range(n-l+1):
8         j=i+l-1; s=sum(f[i:j+1])
9         c[i][j]=min((c[i][x-1] if x>i else 0)+(c[x+1][j] if x<j
10            else 0)+s for x in range(i,j+1))
11         r[i][j]=min(range(i,j+1),key=lambda x:(c[i][x-1] if x>i
12            else 0)+(c[x+1][j] if x<j else 0))
13
14 print("Cost:",c[0][3],"Root:",k[r[0][3]])
15 print("Cost Matrix:",c)
16 print("Root Matrix:",r)
```

Output

Cost: 1.7 Root: C
Cost Matrix: [[0.1, 0.4, 1.1, 1.7], [0, 0.2, 0.8, 1.4], [0, 0, 0.4, 1.0], [0, 0, 0, 0.3]]
Root Matrix: [[0, 1, 2, 2], [0, 1, 2, 2], [0, 0, 2, 2], [0, 0, 0, 3]]

=== Code Execution Successful ===

EXPERIMENT – 16

```
main.py Visualize Run
```

```
1 k=[10,12,16,21]; f=[4,2,6,3]; n=4
2 c=[[0]*n for _ in k]; r=[[0]*n for _ in k]
3
4 for i in range(n): c[i][i]=f[i]; r[i][i]=i
5
6 for l in range(2,n+1):
7     for i in range(n-l+1):
8         j=i+l-1; s=sum(f[i:j+1])
9         c[i][j]=min((c[i][x-1] if x>i else 0)+(c[x+1][j] if x<j
10            else 0)+s for x in range(i,j+1))
11         r[i][j]=min(range(i,j+1),key=lambda x:(c[i][x-1] if x>i
12            else 0)+(c[x+1][j] if x<j else 0))
13
14 print("Cost:",c[0][3])
15 print("Root Matrix:",r)
```

Output

Cost: 26
Root Matrix: [[0, 0, 2, 2], [0, 1, 2, 2], [0, 0, 2, 2], [0, 0, 0, 3]]

=== Code Execution Successful ===

EXPERIMENT – 17

```
main.py Visualize Run
```

```
1 from collections import deque
2
3 def catMouseGame(g):
4     n=len(g); q=deque(); d={}
5     for m in range(1,n):
6         d[m,0]=1; q.append((m,0,0)) # Mouse wins
7         d[2,m,1]=2; q.append((2,m,1)) # Cat wins
8
9     while q:
10         m,c,t=q.popleft()
11         for nm,nc,nt in [(x,c,1) for x in g[m] if t==0] if
12            t==0 else [(m,x,0) for x in g[c] if x]:
13             if (nm,nc,nt) in d: continue
14             d[nm,nc,nt]=d[m,c,t]
15             q.append((nm,nc,nt))
16
17     return d.get((1,2,0),0)
18
19 print(catMouseGame([2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]))
20 print(catMouseGame([1,3],[0],[3],[0,2]))
```

Output

1
1

=== Code Execution Successful ===

EXPERIMENT – 18

```
main.py Visualize Run Output
1 import heapq
2 def maxProb(n, edges, p, start, end):
3     g=[[] for _ in range(n)]
4     for (a,b),w in zip(edges,p):
5         g[a].append((b,w)); g[b].append((a,w))
6     d=[0]*n; d[start]=1
7     q=[(-1,start)]
8     while q:
9         prob,u=heapq.heappop(q); prob=-prob
10        if u==end: return prob
11        for v,w in g[u]:
12            x=prob*w
13            if x>d[v]:
14                d[v]=x
15                heapq.heappush(q,(-x,v))
16    return 0
17 print(maxProb(3,[[0,1],[1,2],[0,2]], [.5,.5,.2],0,2))
18 print(maxProb(3,[[0,1],[1,2],[0,2]], [.5,.5,.3],0,2))
```

0.25
0.3
=== Code Execution Successful ===

EXPERIMENT – 19

```
main.py Visualize Run Output
1 def uniquePaths(m, n):
2     dp = [[0 for j in range(n)] for i in range(m)]
3     for j in range(n):
4         dp[0][j] = 1
5     for i in range(m):
6         dp[i][0] = 1
7     for i in range(1, m):
8         for j in range(1, n):
9             dp[i][j] = dp[i-1][j] + dp[i][j-1]
10    return dp[m-1][n-1]
11 m = 3
12 n = 7
13 print("Number of paths:", uniquePaths(m, n))
14
15 m = 3
16 n = 2
17 print("Number of paths:", uniquePaths(m, n))
```

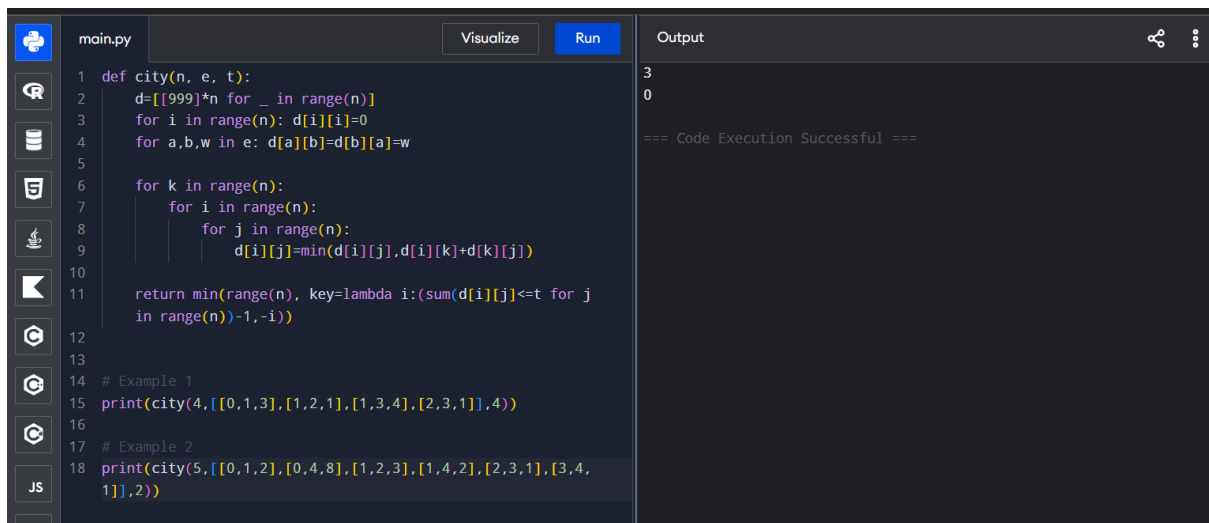
Number of paths: 28
Number of paths: 3
=== Code Execution Successful ===

EXPERIMENT – 20

```
main.py Visualize Run Output
1 def goodPairs(nums):
2     count = 0
3     # Check every possible pair
4     for i in range(len(nums)):
5         for j in range(i + 1, len(nums)):
6
7             # Check if both values are equal
8             if nums[i] == nums[j]:
9                 count += 1
10
11    return count
12 # Example 1
13 nums = [1, 2, 3, 1, 1, 3]
14 print("Number of good pairs:", goodPairs(nums))
15
16 # Example 2
17 nums = [1, 1, 1, 1]
18 print("Number of good pairs:", goodPairs(nums))
```

Number of good pairs: 4
Number of good pairs: 6
=== Code Execution Successful ===

EXPERIMENT – 21



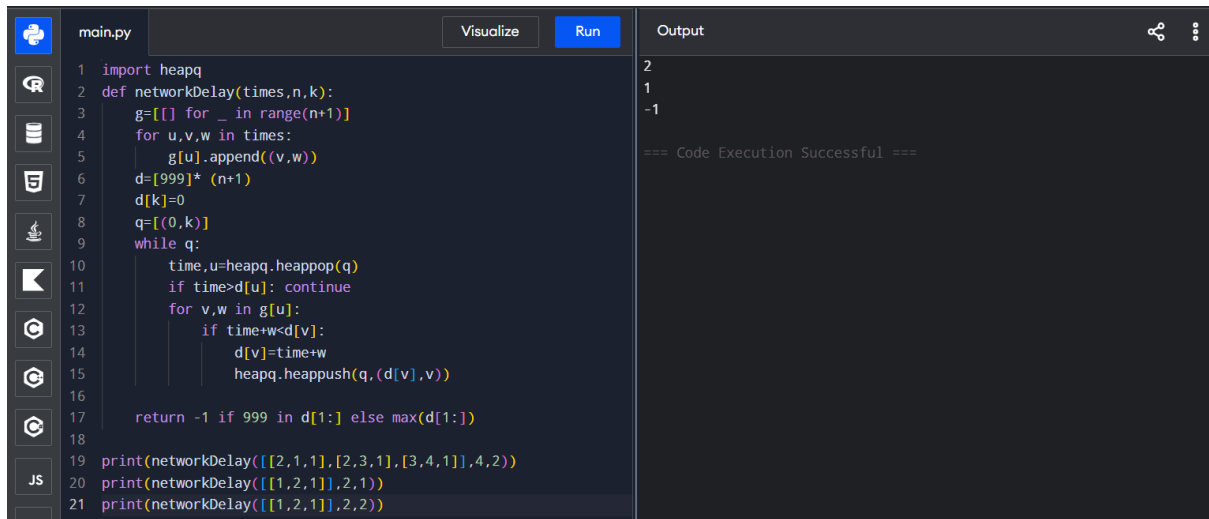
The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'city(n, e, t)' that calculates the shortest path from node 0 to node t in a graph with n nodes and edges e. The edges are represented as a list of (a, b, w) tuples. The function uses Dijkstra's algorithm with a priority queue. Two example calls are provided: 'city(4, [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], 4)' and 'city(5, [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], 2)'. The output window shows the results: 3 and 0, followed by '=== Code Execution Successful ==='.

```
1 def city(n, e, t):
2     d=[[999]*n for _ in range(n)]
3     for i in range(n): d[i][i]=0
4     for a,b,w in e: d[a][b]=d[b][a]=w
5
6     for k in range(n):
7         for i in range(n):
8             for j in range(n):
9                 d[i][j]=min(d[i][j],d[i][k]+d[k][j])
10
11     return min(range(n), key=lambda i:(sum(d[i][j])<=t for j
12         in range(n))-1,-i))
13
14 # Example 1
15 print(city(4,[[0,1,3],[1,2,1],[1,3,4],[2,3,1]],4))
16
17 # Example 2
18 print(city(5,[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]],2))
```

Output

```
3
0
=== Code Execution Successful ===
```

EXPERIMENT – 22



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'networkDelay(times, n, k)' that calculates the network delay from node 0 to node k in a graph with n nodes and edges times. The function uses Dijkstra's algorithm with a priority queue. Two example calls are provided: 'networkDelay([[2,1,1],[2,3,1],[3,4,1]], 4, 2)', 'networkDelay([[1,2,1]], 2, 1)', and 'networkDelay([[1,2,1]], 2, 2)'. The output window shows the results: 2, 1, and -1, followed by '=== Code Execution Successful ==='.

```
1 import heapq
2 def networkDelay(times,n,k):
3     g=[[] for _ in range(n+1)]
4     for u,v,w in times:
5         g[u].append((v,w))
6     d=[999]*(n+1)
7     d[k]=0
8     q=[(0,k)]
9     while q:
10         time,u=heapq.heappop(q)
11         if time>d[u]: continue
12         for v,w in g[u]:
13             if time+w<d[v]:
14                 d[v]=time+w
15                 heapq.heappush(q,(d[v],v))
16
17     return -1 if 999 in d[1:] else max(d[1:])
18
19 print(networkDelay([[2,1,1],[2,3,1],[3,4,1]],4,2))
20 print(networkDelay([[1,2,1]],2,1))
21 print(networkDelay([[1,2,1]],2,2))
```

Output

```
2
1
-1
=== Code Execution Successful ===
```